```
     list.size()-2) && (!find_keyword(list.get(i+1))) &&
(!find_bracket(list.get(i+1).charAt(0)))){
6.      String s1 = return_substring(list.get(i+1));
7.    if(list.get(i-2).contains("class")){
8.    if(i>=3){
9.    if(list.get(i-3).contains("abstract")){
10.   flag =0;
11. }
12.   else if(list.get(i-3).contains("interface")) { flag = 1;}
13.   else if(list.get(i-3).contains("class")) {flag = 1;}
14.    }
15.   if(flag ==0) {
16.     s.add(list.get(i-1)+ "->" + s1);
17.     ArrayList<String> base = hm.get(s1);
18.     if(base==null){
19.   base = new ArrayList<String>();
20.      }
21.     base.add(list.get(i-1));
22.     hm.put(s1,base);
23.   }
24. }
25.   else if(list.get(i-2).contains("interface")){
26.     if(i>=3){
27.   if(list.get(i-3).contains("interface")){flag =1;}
28.   if(list.get(i-3).contains("class")){flag =1;}
29.   if(list.get(i-3).contains("abstract")) {flag = 1;}
30.      }
31.     if(flag ==0) {
32.        s.add(list.get(i-1)+ "->" + s1);
33.        ArrayList<String> base = hm.get(s1);
34.        if(base==null){
35.                base = new ArrayList<String>();
36.                    }
37.                    base.add(list.get(i-1));
38.                    hm.put(s1,base);
39.                }
40.             }
41.          }
42.      }
43.    }
44. }
```

How to visualise the inner, inherited classes

The JTree library is used for building the tree structure. JTree is a swing component that is generally used to build hierarchical tree-like data structures. It has a root node to which we add the mutable nodes.

How to visualise all the classes

To visualise inner classes, store their relations in an ArrayList. If Class A has inner classes, say B and C; and B in turn has an inner class, say D; then the contents in the ArrayList will be of the form: A, A.B, A.B.D, A.C. To visualise the inheritance relations, store them in a HashMap with the key as the parent class and the value as all the base classes (classes that inherit the parent class directly)—for example, if Class A is extended by Class B and Class C. And also if Class B is extended by Class D.

The HashMap data structure representation to store inheritance relations is as follows:

```
A → <B, C>
B → <D>
```

The user interface of the application is divided into three panels. The first panel represents the tree structure of the names of the classes in the package. The second panel represents the inner classes in the package (a class is present in the inner classes tree if it has any inner classes). The code snippet for this is given below:

```
1. public JTree TreeExample(){
2.    Change_List();
3.    s1.clear();
4.    for(int i =0; i< s.size();i++){
5.          s1.add(new DefaultMutableTreeNode(s.get(i)));
6.          if(find_dots(s.get(i)) ==0){
7.                  inner.add(s1.get(i));
8.       }
9.    }
10.   for(int i =0; i< s1.size(); i++){
11.         String str = s1.get(i).getUserObject().toString();
12.         int len = str.length();
13.         int count = find_dots(str);
14.         for(int j =i+1 ; j< s1.size() ; j++){
15.     String str1 = s1.get(j).getUserObject().toString();
16.             int count1 = find_dots(str1);
17.         if((count1 == count+1) && check_substring(str1,
        str)){
18.         s1.get(i).add(s1.get(j));
19.     }
20.   }
21.   }
22.    add(tree1);
23. return tree1;
24.}
```

The third panel represents the inheritance relationships between classes, if there are any. The code snippet for this is given below:

```
1. public JTree TreeBuild() {
2.    Set<String> keys = hm.keySet();
3.    // iterate through the key set and display key and
values
4.    for (String key : keys) {
```

Continued on page no...68